PERSONAL PROJECT

AI 뉴스 쇼츠 제작 자동화

뉴스 수집부터 기사 선정, 대본, 이미지와 영상, 음성, 편집을 자동화한 프로젝트

기간 : 2026.01 - 2026.03
담당 : 기획, 개발, 채널 운영

01 기사 수집
Puppeteer / Sheets

→

02 기사 선정
GPT 평가 / 코드 정렬

→

03 유형과 플롯
태그 / 6개 유형 / 슬롯

→

04 대본과 장면
슬롯 대본 / 장면 JSON

화면 제작 네이버 이미지 검색 → Claude 검증과 선택 → Gemini 이미지 → Veo 영상

음성과 편집 슬롯 대본 → Naver TTS → Whisper 타임스탬프 → 자막 / FFmpeg 편집

공통 연결 키: row_number(기사) + slot index(장면) / 단계별 결과를 JSON과 파일로 전달

PROJECT OVERVIEW

AI가 만든 결과를 다음 작업의 입력으로

단계별 JSON과 식별자를 중심으로 수집, 선정, 대본, 장면, 미디어 생성을 연결했습니다.

수집과 선정

Puppeteer / Google Sheets
GPT 제목 평가와 본문 평가

대본과 장면

태그 → 유형별 플롯 → 슬롯 대본
장면 JSON: 인물, 배경, 미디어 유형

미디어 제작

네이버 검색 / Claude 검증과 선택
Gemini / Veo / 음성 / 자막 편집

직접 설계한 연결 방식

row_number로 기사 정보를 유지하고, 슬롯 번호로 대본과 장면별 파일을 연결했습니다. 생성 결과를 JSON과 디렉터리로 저장해 각 단계의 입력과 출력을 확인할 수 있도록 구성했습니다.

언론사별 수집 결과를 같은 형식으로 정리

제목과 URL을 먼저 모으고, 선정된 후보의 본문을 후속 단계에서 수집합니다.

01 목록 수집

MBC, JTBC, 한겨레 등 수집 모듈을 호출합니다.
언론사마다 try/catch를 나눠 한 곳의 실패에도
다른 언론사의 결과를 모으도록 구성했습니다.

02 본문 수집

media 값에 따라 본문 추출 함수를 선택합니다.
문장 단위 줄바꿈을 정리한 뒤 본문과 본문길이를
기존 기사 객체에 추가합니다.

detail_index.js

```
const formattedText = formatArticleText(fullText);
enrichedArticles.push({
  ...article,
  본문: formattedText,
  본문길이: formattedText.length,
});
```

원본 기사 정보에 본문 추가

기존 기사 객체를 펼쳐 URL과 식별자를 보존
언론사별 본문을 공통 필드로 저장해
다음 평가 단계에서 같은 형식으로 읽기

구현 근거: index.js, detail_index.js, googleSheet.js, get_googlesheet.js. 언론사별 파서 구현 파일은 별도 모듈로 참조

ARTICLE SELECTION

모델은 평가하고, 코드는 정렬과 수량을 결정

제목 평가로 후보를 좁힌 뒤 본문을 평가하는 두 단계로 구성했습니다.

제목 점수화

GPT-4.1-mini로 4개 항목 평가
10개씩 Promise.all 병렬 처리
overall_score는 코드에서 합산

정제와 후보 선택

동일 제목은 Map으로 제거
제거 사유를 별도 JSON에 기록
제작 수량의 2배를 본문 후보로 선택

본문 평가와 최종 선정

본문 기준 5개 항목 평가
HOOK_1 / HOOK_2 생성
NUM_OF_VIDEOS만큼 최종 선택

```
scores.overall_score =
  scores.clickbait_score + scores.emotional_intensity +
  scores.exaggeration_level + scores.information_gap;
scores.row_number = rowNumber;
```

항목별 점수는 GPT가 반환

합계와 기사 식별자는 코드에서 확정

row_number로 원본 기사와 연결

구현 근거: sort_title.js, sort_title_scoring.js, sort_title_duplicate.js, sort_content.js, sort_15.js. 채널별 분류 정책도 별도 단계로 적용

6개 기사 유형에 맞춰 플롯을 미리 정의

core_tag와 secondary_tags를 입력받아, 우선순위 조건문으로 article_class를 결정합니다.

기사 유형	분류 조건 요약	미리 정의한 슬롯 순서
TIMING_FOCUS	DELAY 태그가 있으면 최우선	HOOK → SUMMARY → DELAY → MOTIVE → RESPONSIBILITY → TIMING_Q
CONFLICT_FOCUS	REACTION / CONFLICT / POWER_STRUGGLE	HOOK → A → B → CLASH → ESCALATION → JUDGMENT → POLL_Q
CONTROVERSY_FOCUS	SCANDAL / MEDIA_ISSUE / PUBLIC_OPINION	HOOK → ISSUE → FACT → RESPONSE → DOUBT → SUSPENSE_Q
RESPONSIBILITY_FOCUS	EXPLANATION / HISTORICAL_REFERENCE	HOOK → CONTEXT → EXPLANATION → STANDARD → JUDGMENT → RESPONSIBILITY_Q
PROCESS_FOCUS	core_tag: LEGAL / INVESTIGATION / RULING	HOOK → STATUS → PROCEDURE → PROBLEM → NEXT → LEGITIMACY_Q
FACT_FOCUS	위 조건에 해당하지 않으면 기본 유형	HOOK → SETUP → DELAY → SUSPENSE_Q → CTA → ANSWER

설계 의도: 유형과 문장 순서를 코드로 고정하고, 각 슬롯에 들어갈 대본을 생성 단계에 전달

구현 근거: get_slot.js의 classifyArticleClass, structureMap. 조건은 위에서부터 평가되며 core / secondary 태그의 적용 범위는 코드 기준.

플롯의 각 슬롯에 역할과 작성 규칙을 전달

슬롯은 영상의 한 구간입니다. 기사 유형에 따라 순서, 역할, 작성 지침을 정한 뒤 대본 생성 함수를 호출합니다.

갈등형 예시

HOOK → A → B → CLASH → ESCALATION → JUDGMENT → POLL_Q

```
B: {
  role: "반대편 입장을 동일한 비중으로 요약한다.",
  guidelines: "A와 톤·길이 균형 유지.",
  intent: "대립 구조 형성"
}

// 각 슬롯에 role / guidelines / intent 주입
```

role: 이 구간에서 설명할 내용

guidelines: 문장 작성 시 지킬 규칙

intent: 해당 구간의 목적

같은 갈등형이라도 기사 본문을 함께 전달해 정해진 슬롯에 맞는 대본을 작성하도록 구성

tagged_articles.json → slotted_articles.json → processScripts(...) → scripted_articles.json

구현 근거: get_slot.js의 slotRoleMap, processSlots / write_slot.js. 역할과 지침은 생성 입력이며 결과 준수 여부의 자동 검증과는 구분됩니다.

장면 JSON을 검색과 생성의 공통 입력으로 사용

scene_describer는 슬롯별 인물과 배경, 장면 설명을 정해 후속 단계에 전달합니다.

```
{
  "row_number": ${article.row_number},
  "slots": [
    {
      "index": number,
      "media_type": "image" | "video",
      "주인공 수": "1" | "2",
      "주인공1": "실명(한국어)",
      "주인공2": "실명(한국어) 또는 null",
      "배경": "실제 장소명(한국어)",
      "장면 설명": "protagonist1 / protagonist2로 묘사"
    }
  ]
}
```

필드별 사용처

row_number / index
기사와 장면별 파일을 연결

주인공1 / 주인공2 / 배경
후속 이미지 검색에 사용할 대상 정보

media_type / 장면 설명
이미지 또는 영상 생성 단계의 입력

프롬프트 규칙
슬롯 수와 media_type 유지
장면 속 인물은 protagonist1 / 2로 지칭

출력 스키마: 타입과 자리표시자를 포함한 프롬프트 명세입니다. 구현 근거: scene_describer.js / scene_describe.js.

네이버 검색 결과를 Claude로 검증하고 참조 이미지로 사용

장면 JSON의 인물과 배경 정보를 바탕으로 이미지를 검색하고, 생성에 사용할 후보를 선택했습니다.

검색 대상
인물 / 배경 정보

→

네이버 검색
기사별 후보 수집

→

Claude 검토
후보 검증과 선택

→

생성 입력
인물 / 배경 참조

```
// 기사별 검색 작업 실행
spawn("node", [naverSearchPath, tempFile], {
  stdio: "inherit",
  env: { ...process.env, TEMP_INPUT_FILE: tempFile }
});

// 모든 기사 검색이 끝난 뒤 이미지 선택 실행
spawn("node", [imageSelectorPath], {
  stdio: "inherit"
});
```

1. row별 JSON을 임시 파일로 저장
2. naver_search.js에 경로를 전달
3. 모든 검색 성공 후 image_selector.js 실행

선택된 결과의 파일 경로

image_selected/row_{n}/slot_{i}/

protagonist1.jpg / protagonist2.jpg

background.jpg

구현 근거: run_naver.js, create_image.py. Claude 검증과 선택은 사용자 운영 설명 기준이며, 상세 판정 조건은 image_selector.js 내부

이미지를 만든 뒤 필요한 장면만 영상으로

정지 이미지와 영상 생성을 슬롯 단위로 분리했습니다.

Gemini 이미지 생성

Claude가 선택한 인물과 배경 이미지를 참조 입력으로 사용
gemini-2.5-flash-image / 9:16 이미지 생성
API 오류 시 최대 3회 시도, 대기 시간 증가
row와 slot 디렉터리에 generated.jpg 저장

Veo 영상 생성

media_type이 video인 슬롯만 처리
google/veo-3.1-fast / 4초 / 720p / 9:16
generate_audio: false로 내레이션과 분리
generated.mp4가 있으면 재생성 없이 건너뜀

create_video.js

```
if (slot.media_type !== 'video') continue;
try {
  await fs.access(videoPath);
  totalSuccess++;
  continue;
} catch {}
```

완료된 영상은 재생성 생략

영상 슬롯만 처리하고 기존 파일을 확인
완료된 슬롯은 건너뛰어 중복 API 호출을 줄이기
FACT_FOCUS의 4번 슬롯도 별도 조건으로 생략

구현 근거: run_naver.js, create_image.py, create_video.js. 이미지 검색과 선택은 작업 프로세스 호출

음성 길이에 맞춰 자막과 편집을 연결

대본의 슬롯 경계를 음성 생성 입력에도 유지했습니다.

코드에서 확인한 음성 처리

슬롯별 텍스트를 정리해 TTS 입력 JSON 생성
TTS_NUM 설정으로 1~6번 worker 선택
음성 생성 성공 후 tts_sync.js 순차 실행
하위 프로세스 실패 시 후속 단계 중단

운영 과정에서 적용한 방식

Naver TTS로 내레이션 생성
Whisper로 음성을 분석해 타임스탬프 추출
문장별 자막 표시 시점 조정
FFmpeg 편집과 채널 업로드 자동화

run_tts.js

```
await runCommand(
  "node", [ttsWorkerFile], ttsStyleDir
);
await runCommand(
  "node", ["tts_sync.js"], utilsDir
);
```

음성 생성 후 동기화 실행

runCommand는 종료 코드가 0일 때 resolve
실패하면 reject되어 다음 await로 진행하지 않게
생성 음성을 다시 분석해 자막 시점을 보완

구현 근거: prepare_tts.js, run_tts.js. Naver TTS, Whisper, FFmpeg

ENGINEERING DECISIONS

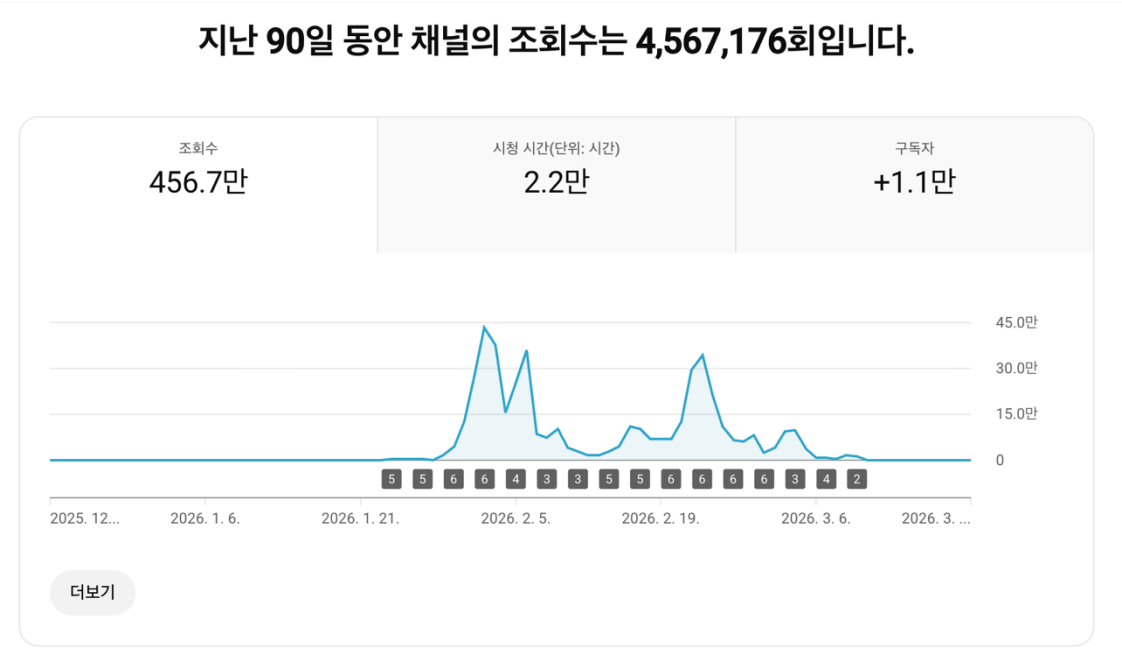
AI 판단과 반복 처리 규칙을 나누어 구현

모델이 맡을 작업과 코드가 확정할 작업을 구분했습니다.

점수 계산	제목의 항목별 점수는 GPT가 평가	합산, 내림차순 정렬, 선정 수량은 코드가 결정
데이터 연결	GPT 출력은 JSON으로 파싱	row_number는 원본 값으로 주입해 기사 연결 유지
반복 실행	이미지 생성 실패는 재시도	기존 영상 파일은 건너뛰어 중복 생성 방지
작업 순서	TTS worker를 하위 프로세스로 실행	종료 코드가 0일 때 동기화 단계로 진행

성과

실제 채널 분석 화면으로 운영 결과를 확인했습니다.



PROJECT TAKEAWAYS

생성형 AI를 반복 운영 가능한 작업으로 연결

개인 프로젝트에서 직접 수행한 개발 경험과 다음 개선 방향입니다.

이 프로젝트에서 수행한 일

뉴스 수집과 Google Sheets 연동
GPT 평가, 후보 정제, 대본과 장면 단계 연결
Gemini와 Veo API 기반 미디어 생성
음성 및 편집 파이프라인 운영

다음 개선 방향

점수 범위 검증 강화
실패 상태를 후속 단계와 분리해 재실행 관리

AI 서비스 개발에 적용할 수 있는 경험

서로 다른 API의 결과를 공통 데이터로 연결하고, 실패와 재실행을 고려해 처리 단계를 구성했습니다. 실제 채널을 운영하며 모델 출력이 후속 작업에서 사용될 수 있는 형태로 전달되어야 한다는 점을 확인했습니다.